

Códigos de Control de Errores

(Unidad 5)

Guerra Jorge, Pesce Agustín, Villar Federico

29 de agosto de 2026

Resumen

Las unidades anteriores dejaron cerrado el marco de los códigos de bloque lineales, pero también expusieron su límite práctico: la decodificación por tabla de síndromes requiere 2^{n-k} entradas y deja de ser realizable apenas la redundancia crece. Esta unidad resuelve el problema imponiendo al código una simetría adicional —la invariancia frente a desplazamientos cíclicos— que permite reemplazar tablas por circuitos secuenciales. Se establece primero la correspondencia entre n -tuplas y polinomios, bajo la cual el desplazamiento cíclico se traduce en la multiplicación por X módulo $X^n + 1$, y sobre ella se demuestran los cuatro teoremas que estructuran la teoría: la existencia y unicidad del polinomio generador $g(X)$, de grado $n - k$; la caracterización de las palabras de código como los múltiplos de $g(X)$; el hecho de que $g(X)$ divida a $X^n + 1$; y su recíproco, que convierte el diseño de códigos cíclicos en un problema de factorización. Se muestra que la codificación sistemática se reduce a una única división de polinomios, realizable con un registro de desplazamiento de $n - k$ etapas cuya complejidad crece linealmente con la redundancia, y se traza su funcionamiento ciclo a ciclo sobre el código de Hamming cíclico $(7, 4)$. El cálculo del síndrome emplea el mismo circuito, y la propiedad de desplazamiento —que el síndrome de una palabra desplazada se obtiene con un ciclo de reloj adicional— sostiene los dos procedimientos de decodificación que se desarrollan: el decodificador de Meggitt, que reduce el reconocimiento de síndromes a un subconjunto mucho menor que el conjunto de líderes de coset, y la decodificación por captura de errores, que en el caso favorable prescinde por completo de lógica de reconocimiento. Se demuestra además que todo código cíclico detecta las ráfagas de longitud hasta $n - k$, y se cuantifica la fracción no detectada para ráfagas más largas. La unidad cierra con las familias de interés práctico: los códigos de Hamming cíclicos generados por polinomios primitivos, el código de Golay $(23, 12)$ en su presentación cíclica y perfecta, los códigos acortados que dan origen a los CRC normalizados, los códigos producto cíclicos y los quasi-cíclicos. Una advertencia recorre todo el desarrollo: la construcción es sistemática, pero la elección del polinomio generador no lo es, y para una misma redundancia conviven códigos de distancias mínimas muy diferentes. Las figuras del apunte se generan con los scripts de Python que lo acompañan.

1. Del código lineal al código cíclico

Las unidades precedentes dejaron cerrado el marco de los códigos de bloque lineales. Un código (n, k) queda definido por su matriz generadora G o, equivalentemente, por su matriz de paridad H ; la codificación es un producto matriz-vector; el síndrome $\mathbf{s} = \mathbf{r}H^T$ depende únicamente del patrón de error y no de la palabra transmitida; y la decodificación consiste en asociar a cada síndrome el líder de coset correspondiente. Ese marco es completo desde el punto de vista teórico.

Desde el punto de vista de la implementación, en cambio, presenta dos obstáculos serios. El primero es el costo de la codificación: el producto $\mathbf{u}G$ requiere del orden de $k(n-k)$ operaciones y una memoria que almacene los $k(n-k)$ bits de la submatriz de paridad. El segundo, y mucho más grave, es el costo de la decodificación: la tabla que asocia síndromes con líderes de coset tiene 2^{n-k} entradas. Para el código de Hamming $(7, 4)$ son ocho entradas y la tabla se dibuja en una transparencia; para un código con $n-k = 32$ bits de redundancia son más de cuatro mil millones de entradas, y el procedimiento deja de ser realizable.

La salida no consiste en abandonar la estructura lineal, sino en **agregarle** estructura. Si el conjunto de palabras de código se elige de modo que satisfaga una propiedad de simetría adicional, esa simetría puede explotarse para reemplazar tablas por circuitos secuenciales. La propiedad que se impone en esta unidad es la invariancia frente a desplazamientos cíclicos, y la clase de códigos que resulta –los códigos cíclicos– es, con diferencia, la más importante de la práctica: los códigos BCH, los Reed-Solomon y los CRC de uso industrial son todos cíclicos o derivados de códigos cíclicos.

1.1. Definición

Sea $\mathbf{v} = (v_0, v_1, \dots, v_{n-1})$ una n -tupla sobre $GF(2)$. Se llama *desplazamiento cíclico* (cyclic shift) de $\mathbf{v}$ a la n -tupla

$$\mathbf{v}^{(1)} = (v_{n-1}, v_0, v_1, \dots, v_{n-2}) \quad (1)$$

obtenida moviendo cada componente una posición hacia la derecha y llevando la última componente al primer lugar. Aplicando la operación i veces se obtiene el desplazamiento de orden i , que se denota $\mathbf{v}^{(i)}$.

Definición. Un código de bloque lineal (n, k) se dice *cíclico* si el desplazamiento cíclico de toda palabra de código es también una palabra de código.

Conviene observar de inmediato que la condición es fuerte. Un código lineal cualquiera puede tener cualquier conjunto de 2^k palabras cerrado bajo la suma; exigir además el cierre bajo desplazamientos restringe considerablemente las posibilidades y, como se verá, permite describir el código entero mediante un único polinomio de grado $n-k$ en lugar de una matriz de $k \times n$ bits.

1.2. Representación polinómica

El desplazamiento cíclico se maneja con incomodidad en notación vectorial. La herramienta natural es la representación polinómica: a la n -tupla $\mathbf{v}$ se le asocia el polinomio

$$v(X) = v_0 + v_1X + v_2X^2 + \cdots + v_{n-1}X^{n-1} \quad (2)$$

de grado a lo sumo $n - 1$ y coeficientes en $GF(2)$. La correspondencia entre n -tuplas y polinomios de grado menor que n es biunívoca, y la suma de vectores se corresponde con la suma de polinomios. Lo que se gana es una descripción algebraica del desplazamiento. Multiplicando (2) por X se obtiene

$$Xv(X) = v_0X + v_1X^2 + \cdots + v_{n-2}X^{n-1} + v_{n-1}X^n \quad (3)$$

que no es un polinomio admisible, porque su grado puede llegar a n . Sumando y restando v_{n-1} –operaciones idénticas sobre $GF(2)$ – resulta

$$Xv(X) = v_{n-1}(X^n + 1) + v^{(1)}(X) \quad (4)$$

donde $v^{(1)}(X)$ es exactamente el polinomio asociado a la n -tupla (1). La igualdad (4) dice que $v^{(1)}(X)$ es el resto de dividir $Xv(X)$ por $X^n + 1$:

$$v^{(1)}(X) = R_{X^n+1}\{Xv(X)\} \quad (5)$$

donde, en lo que sigue, $R_{d(X)}\{f(X)\}$ denota el resto de dividir $f(X)$ por $d(X)$. La notación no es estándar –Lin & Costello escriben la operación en palabras–, pero abrevia considerablemente los enunciados de esta unidad, en los que casi todo es un resto. De manera general, el desplazamiento de orden i es $R_{X^n+1}\{X^i v(X)\}$. La Figura 1 muestra las dos lecturas de esta identidad: la geométrica, sobre un anillo de n posiciones, y la algebraica, como multiplicación sucesiva por X con la reducción $X^n \equiv 1$.

(b) Successive shifts as multiplication by X

(a) Cyclic shift on the ring of $n = 7$ positions

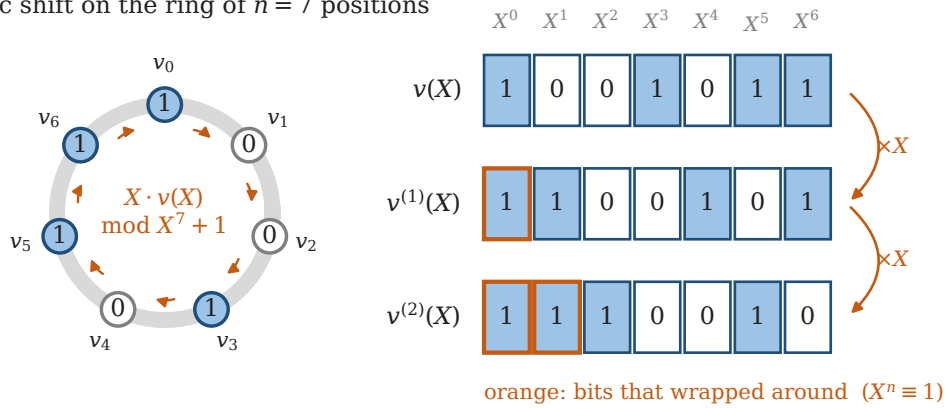


Figura 1: Desplazamiento cíclico de una palabra de longitud $n = 7$. A la izquierda, la interpretación sobre el anillo de posiciones: cada componente avanza un lugar y la de mayor orden vuelve al principio. A la derecha, la misma operación como multiplicación por X módulo $X^n + 1$; en naranja, las componentes que dieron la vuelta.

El aparato algebraico queda entonces montado sobre el anillo de polinomios módulo $X^n + 1$. Todo lo que sigue en la unidad son consecuencias de esta estructura.

2. El polinomio generador

El resultado central de la teoría de códigos cíclicos es que un único polinomio determina el código completo. Se lo enuncia y demuestra en cuatro pasos.

Teorema 1 (Teoremas 5.1, 5.2 y 5.4 de Lin & Costello). *Sea C un código cíclico (n, k) no trivial. Existe un único polinomio no nulo de grado mínimo en C . Ese polinomio, denotado $g(X)$, tiene término independiente $g_0 = 1$ y grado $n - k$.*

Demostración. Sea $g(X) = g_0 + g_1X + \cdots + g_rX^r$ un polinomio de código no nulo de grado mínimo r .

Término independiente. Si fuera $g_0 = 0$, entonces $g(X) = X\tilde{g}(X)$ con $\tilde{g}(X)$ de grado $r - 1$. Como $r \leq n - 1$, el desplazamiento cíclico de orden $n - 1$ de $g(X)$ no produce acarreo módulo $X^n + 1$ y coincide con $\tilde{g}(X)$, que sería entonces un polinomio de código no nulo de grado menor que r , en contra de la minimalidad. Luego $g_0 = 1$.

Unicidad. Si $g(X)$ y $g'(X)$ fueran dos polinomios de código distintos de grado r , ambos mónicos sobre $GF(2)$, su suma $g(X) + g'(X)$ sería un polinomio de código no nulo de grado estrictamente menor que r , nuevamente en contra de la minimalidad.

Grado. Los $n - r$ polinomios $g(X), Xg(X), \dots, X^{n-r-1}g(X)$ tienen grado a lo sumo $n - 1$, de modo que son desplazamientos cíclicos de $g(X)$ sin acarreo y, por lo tanto, palabras de código. Son además linealmente independientes, porque tienen grados distintos. Por el Teorema 2, que se demuestra a continuación, toda palabra de código es una combinación lineal de ellos, de modo que la dimensión del código es $n - r$. Como la dimensión es k , resulta $r = n - k$.

Teorema 2 (Teorema 5.3). *Un polinomio $v(X)$ de grado menor que n es una palabra de código si y sólo si es múltiplo de $g(X)$.*

Demostración. Sea $v(X)$ una palabra de código. Dividiendo por $g(X)$ se obtiene $v(X) = a(X)g(X) + b(X)$ con $\deg b(X) < \deg g(X)$. Los productos $X^i g(X)$ con $i \leq n - 1 - \deg g(X)$ son palabras de código, según se vio en la demostración anterior; por linealidad, $a(X)g(X)$ también lo es, y entonces $b(X) = v(X) + a(X)g(X)$ es una palabra de código de grado menor que el de $g(X)$. Por minimalidad de $g(X)$, debe ser $b(X) = 0$.

Recíprocamente, si $v(X) = u(X)g(X)$ con $\deg v(X) < n$, entonces $v(X)$ es una combinación lineal de los polinomios $X^i g(X)$, todos ellos palabras de código, y por linealidad es una palabra de código.

El Teorema 2 tiene una consecuencia práctica inmediata: **verificar si una palabra recibida pertenece al código se reduce a una división de polinomios.** No hace falta ninguna matriz.

Teorema 3 (Teorema 5.5). *El polinomio generador de un código cíclico (n, k) es un divisor de $X^n + 1$.*

Demostración. Multiplicando $g(X)$, de grado $n - k$, por X^k se obtiene un polinomio de grado n . Por (4) aplicada k veces,

$$X^k g(X) = (X^n + 1) + g^{(k)}(X) \quad (6)$$

donde $g^{(k)}(X)$ es el desplazamiento cíclico de orden k de $g(X)$ y, por lo tanto, una palabra de código. Por el Teorema 2 existe $a(X)$ tal que $g^{(k)}(X) = a(X)g(X)$, de donde

$$X^n + 1 = [X^k + a(X)] g(X) \quad (7)$$

es decir, $g(X)$ divide a $X^n + 1$.

Teorema 4, recíproco (Teorema 5.6). *Si $g(X)$ es un divisor de $X^n + 1$ de grado $n - k$, entonces $g(X)$ genera un código cíclico (n, k) .*

Demostración. Considérese el conjunto C de los 2^k polinomios $v(X) = u(X)g(X)$ con $\deg u(X) < k$. El conjunto es cerrado bajo la suma, de modo que es un código lineal (n, k) . Resta ver que es cíclico. Sea $v(X) = u(X)g(X) \in C$. Por (4),

$$v^{(1)}(X) = X v(X) + v_{n-1}(X^n + 1) \quad (8)$$

y, escribiendo $X^n + 1 = h(X)g(X)$, resulta

$$v^{(1)}(X) = [X u(X) + v_{n-1}h(X)] g(X) \quad (9)$$

que es un múltiplo de $g(X)$ de grado menor que n y, por lo tanto, un elemento de C .

2.1. Consecuencia constructiva

Los Teoremas 3 y 4 juntos resuelven un problema que la Unidad 4 había dejado abierto: cómo **construir** códigos con buenos parámetros en lugar de limitarse a analizarlos. El procedimiento es directo:

1. Se fija la longitud n .
2. Se factoriza $X^n + 1$ en factores irreducibles sobre $GF(2)$.
3. Cada producto de un subconjunto de esos factores es un polinomio generador válido, y su grado determina la redundancia $n - k$.

Si $X^n + 1$ tiene f factores irreducibles distintos, hay $2^f - 2$ códigos cíclicos no triviales de longitud n . Para $n = 15$ la factorización es

$$X^{15} + 1 = (1+X)(1+X+X^2)(1+X^3+X^4)(1+X+X^4)(1+X+X^2+X^3+X^4) \quad (10)$$

con cinco factores, de modo que existen $2^5 - 2 = 30$ códigos cíclicos no triviales de longitud 15. La Figura 2 muestra la construcción y el catálogo completo.

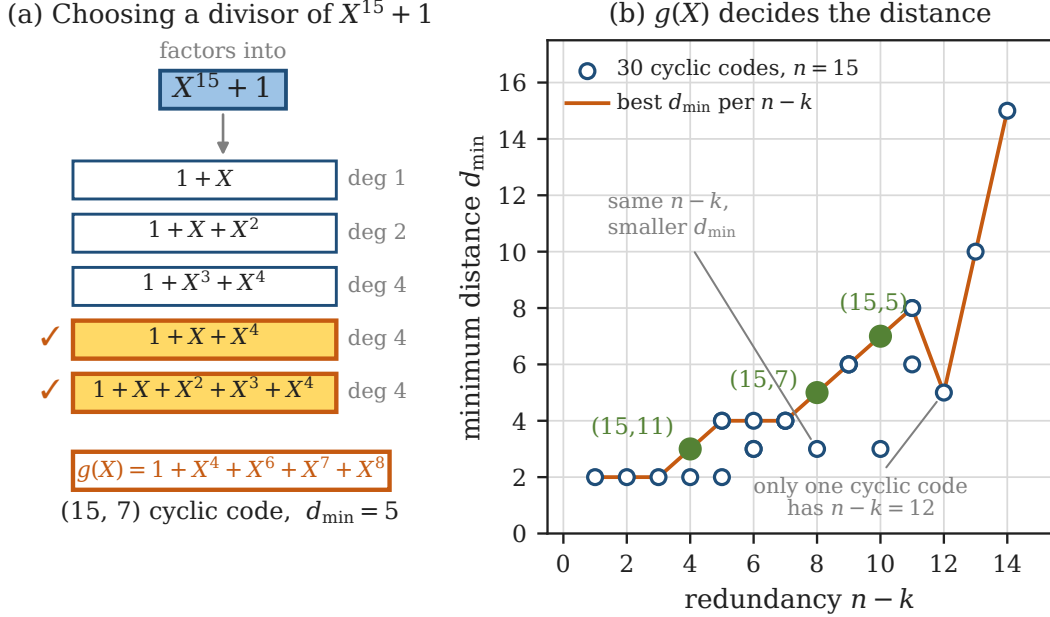


Figura 2: Construcción de códigos cíclicos de longitud 15. A la izquierda, la factorización de $X^{15} + 1$ y la elección de dos de sus factores como polinomio generador. A la derecha, los 30 códigos cíclicos no triviales de esa longitud, con la distancia mínima calculada por enumeración exhaustiva de las palabras.

El panel derecho de la figura contiene la advertencia más importante de esta sección: **la redundancia no determina la distancia mínima**. Con $n - k = 8$ existen tres códigos cíclicos de longitud 15, y sus distancias mínimas son 5, 5 y 3. Elegir mal el polinomio generador cuesta la mitad de la capacidad correctora sin ahorrar un solo bit de redundancia.

El comportamiento no monótono de la curva de mejores distancias merece un comentario, porque a primera vista resulta desconcertante: al pasar de $n - k = 11$ a $n - k = 12$ la distancia mínima alcanzable **cae** de 8 a 5, para volver a subir a 10 y 15 en las dos redundancias siguientes. No hay error en el cálculo; lo que ocurre es que el catálogo de longitud 15 no ofrece la misma variedad para toda redundancia. La única descomposición de 12 como suma de grados de factores de (10) es $4 + 4 + 4$, de modo que **existe un solo código cíclico con $n - k = 12$** , el generado por el producto de los tres factores de grado cuatro. Su polinomio de paridad es $h(X) = 1 + X^3$, lo que impone a toda palabra la condición $v_i = v_{i+3}$: las palabras son un patrón de tres bits repetido cinco veces, su peso es cinco veces el peso del patrón, y la distancia mínima es por lo tanto 5. Con $n - k = 11$, en cambio, hay tres códigos disponibles, y el mejor de ellos –el de polinomio de paridad primitivo $h(X) = 1 + X + X^4$, dual del Hamming (15, 11)– es el código simplex, cuyas quince palabras no nulas tienen todas peso 8.

La moraleja es que la estructura cíclica no sólo restringe qué códigos existen, sino que puede dejar huecos: para ciertas combinaciones de n y $n - k$ el único código cíclico disponible es malo, y conviene entonces cambiar de longitud o acortar un código de otra redundancia. La Tabla 1 recoge los códigos de mejor distancia para cada redundancia.

Tabla 1: Mejores códigos cíclicos de longitud 15 para cada valor de la redundancia. Tabla generada con `codigo_ciclico.py`.

$n - k$	k	$d_{\min}$	Polinomio generador
4	11	3	$1 + X + X^4$
5	10	4	$1 + X + X^3 + X^5$
6	9	4	$1 + X^2 + X^3 + X^4 + X^6$
7	8	4	$1 + X + X^3 + X^7$
8	7	5	$1 + X + X^2 + X^4 + X^8$
9	6	6	$1 + X + X^4 + X^5 + X^6 + X^9$
10	5	7	$1 + X + X^2 + X^4 + X^5 + X^8 + X^{10}$
11	4	8	$1 + X + X^2 + X^3 + X^5 + X^7 + X^8 + X^{11}$

2.2. Matriz generadora y matriz de paridad

Aunque el objetivo de la unidad es prescindir de las matrices, resulta útil verificar que el código cíclico es, en efecto, un código lineal ordinario y exhibir sus matrices. Como los k polinomios $g(X)$, $Xg(X)$, $\dots$, $X^{k-1}g(X)$ generan el código y son linealmente independientes, la matriz generadora se obtiene escribiéndolos como filas:

$$G = \begin{pmatrix} g_0 & g_1 & g_2 & \cdots & g_{n-k} & 0 & \cdots & 0 \\ 0 & g_0 & g_1 & \cdots & g_{n-k-1} & g_{n-k} & \cdots & 0 \\ \vdots & & \ddots & & & & \ddots & \vdots \\ 0 & \cdots & 0 & g_0 & g_1 & \cdots & & g_{n-k} \end{pmatrix} \quad (11)$$

Es una matriz *de Toeplitz*: cada fila es la anterior desplazada un lugar. Nótese que la matriz completa queda especificada por sus $n - k + 1$ coeficientes g_i , no por sus kn entradas.

El polinomio $h(X) = (X^n + 1)/g(X)$, de grado k , recibe el nombre de *polinomio de paridad* (parity polynomial). Puesto que $g(X)h(X) = X^n + 1$, para toda palabra de código $v(X) = u(X)g(X)$ se cumple

$$v(X)h(X) = u(X)g(X)h(X) = u(X)(X^n + 1) \equiv 0 \pmod{X^n + 1} \quad (12)$$

Igualando a cero los coeficientes de $X^k, X^{k+1}, \dots, X^{n-1}$ del producto $v(X)h(X)$ se obtienen $n - k$ ecuaciones de paridad, cuyos coeficientes forman las filas de la matriz de paridad. En términos polinómicos, el código dual de un código cíclico es el código cíclico generado por el polinomio recíproco $X^k h(X^{-1})$.

3. Codificación

3.1. Codificación sistemática

La codificación no sistemática es inmediata: al mensaje $u(X)$ le corresponde la palabra $v(X) = u(X)g(X)$. Tiene el inconveniente de que el mensaje no aparece explícitamente en la palabra transmitida, lo que obliga al receptor a dividir para recuperarlo. La forma sistemática se obtiene con un artificio estándar.

Se desplaza el mensaje $n - k$ posiciones hacia los términos de mayor orden, multiplicándolo por X^{n-k} , y se lo divide por el polinomio generador:

$$X^{n-k}u(X) = a(X)g(X) + b(X), \quad \deg b(X) < n - k \quad (13)$$

Reordenando, y recordando que sobre $GF(2)$ sumar y restar es lo mismo,

$$b(X) + X^{n-k}u(X) = a(X)g(X) \quad (14)$$

El miembro izquierdo es un múltiplo de $g(X)$ de grado menor que n y, por el Teorema 2, una palabra de código. Sus k coeficientes de mayor orden reproducen el mensaje y sus $n - k$ coeficientes de menor orden son los bits de paridad. Formalmente,

$$v(X) = \underbrace{b_0 + b_1X + \cdots + b_{n-k-1}X^{n-k-1}}_{\text{paridad}} + \underbrace{u_0X^{n-k} + \cdots + u_{k-1}X^{n-1}}_{\text{mensaje}} \quad (15)$$

con $b(X) = R_{g(X)}\{X^{n-k}u(X)\}$. La codificación sistemática de un código cíclico es, por lo tanto, **una sola división de polinomios**.

3.2. Realización con registros de desplazamiento

La división de polinomios sobre $GF(2)$ admite una realización directa en hardware mediante un registro de desplazamiento realimentado (linear feedback shift register, LFSR) cuyas conexiones de realimentación reproducen los coeficientes del divisor. El circuito correspondiente a $g(X)$ se muestra en la Figura 3.

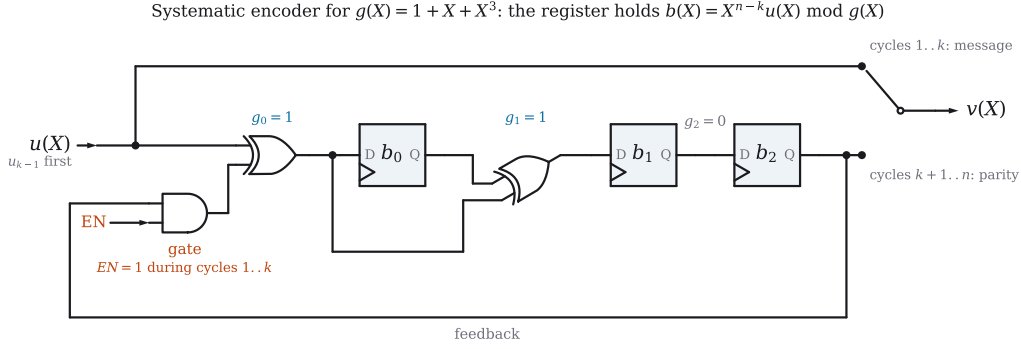


Figura 3: Codificador sistemático de un código cíclico (n, k) . El registro tiene $n - k$ etapas y una conexión de realimentación por cada coeficiente no nulo de $g(X)$. Durante los primeros k ciclos la compuerta está habilitada y el mensaje sale directamente por la derivación superior; durante los $n - k$ ciclos restantes la compuerta se inhibe y el contenido del registro, que es la paridad, se descarga a la salida. Adaptada de las Figuras 5.1 y 5.2 de Lin & Costello, *Error Control Coding*, 2ª ed.

El circuito consta de $n - k$ etapas de memoria y de a lo sumo $n - k - 1$ sumadores módulo dos, es decir, de una cantidad de recursos que crece **linealmente** con la redundancia. La comparación con el codificador matricial genérico, de complejidad $k(n - k)$, es el primer beneficio concreto de la estructura cíclica.

La operación se divide en dos fases:

- **Ciclos 1 a k .** La compuerta está habilitada y los bits del mensaje ingresan al registro empezando por el de mayor orden, u_{k-1} . Simultáneamente, esos mismos bits se copian a la salida a través de la derivación directa, lo que produce la parte sistemática de la palabra.
- **Ciclos $k + 1$ a n .** La compuerta se inhibe, con lo que se corta la realimentación, y el conmutador de salida pasa a tomar el contenido del registro. Los $n - k$ bits que se descargan son los coeficientes de $b(X)$.

Al cabo de n ciclos de reloj la palabra de código está completa. La Figura 4 traza el proceso sobre un ejemplo concreto.

3.3. Codificación mediante el polinomio de paridad

El circuito anterior no es la única realización posible. Las $n - k$ ecuaciones de paridad (12) pueden escribirse, usando que $h_k = 1$, como una *ecuación en diferencias*

$$v_{n-k-j} = \sum_{i=0}^{k-1} h_i v_{n-i-j}, \quad 1 \leq j \leq n - k \quad (16)$$

que determina cada bit de paridad a partir de los k bits precedentes. La realización correspondiente es un registro de k **etapas** cuyas conexiones de realimentación reproducen los coeficientes de $h(X)$, en lugar de los de $g(X)$.

La existencia de las dos alternativas tiene una consecuencia práctica que conviene retener: conviene elegir el circuito cuyo registro sea más corto. Para códigos con más

bits de paridad que de mensaje –es decir, con $n - k > k$, como el $(7, 3)$ o el $(23, 12)$ – el codificador de k etapas basado en $h(X)$ es más económico; en el caso contrario, que es el habitual en los códigos de alta tasa, conviene el de $n - k$ etapas basado en $g(X)$.

3.4. El código de Hamming cíclico $(7, 4)$

Como $X^7 + 1 = (1 + X)(1 + X + X^3)(1 + X^2 + X^3)$, la elección $g(X) = 1 + X + X^3$ produce un código cíclico $(7, 4)$. Su distancia mínima es 3 y su distribución de pesos es $A_0 = 1$, $A_3 = 7$, $A_4 = 7$, $A_7 = 1$: se trata del código de Hamming $(7, 4)$ de la Unidad 4, ahora presentado como código cíclico. La Tabla 2 lista las dieciséis palabras.

Tabla 2: Palabras del código cíclico $(7, 4)$ generado por $g(X) = 1 + X + X^3$, en forma sistemática $\mathbf{v} = (b_0 \ b_1 \ b_2 \ u_0 \ u_1 \ u_2 \ u_3)$. Tabla generada con `codigo_ciclico.py`.

Mensaje	Paridad	Palabra	Mensaje	Paridad	Palabra
0000	000	0000000	1000	110	1101000
0001	101	1010001	1001	011	0111001
0010	111	1110010	1010	001	0011010
0011	010	0100011	1011	100	1001011
0100	011	0110100	1100	101	1011100
0101	110	1100101	1101	000	0001101
0110	100	1000110	1110	010	0101110
0111	001	0010111	1111	111	1111111

Puede comprobarse sobre la tabla que el desplazamiento cíclico de cualquier palabra vuelve a ser una palabra: las siete palabras de peso 3 son los siete desplazamientos de una sola de ellas, y otro tanto ocurre con las de peso 4. Esta observación no es anecdótica: la propiedad cíclica organiza el código en *órbitas* de desplazamiento, y esa estructura es la que el decodificador explotará.

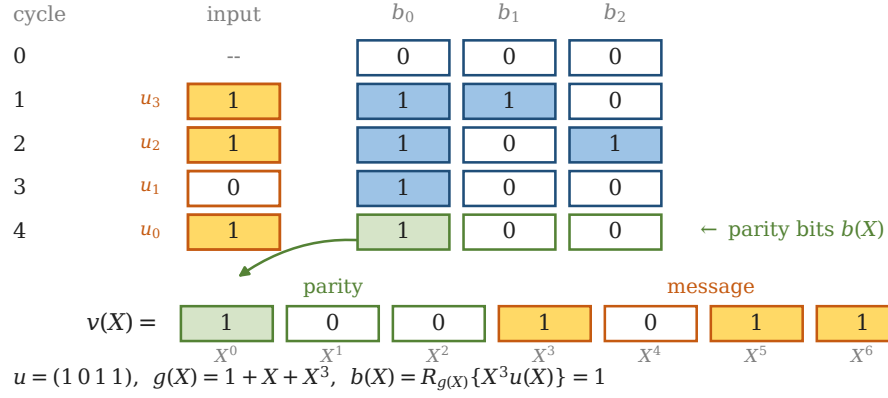


Figura 4: Traza del codificador sistemático del código (7,4) para el mensaje $\mathbf{u} = (1011)$. Los bits ingresan empezando por u_3 ; al cabo de $k = 4$ ciclos el registro contiene la paridad $\mathbf{b} = (100)$, que se antepone al mensaje para formar la palabra transmitida. Los valores reproducen el Ejemplo 5.5 de Lin & Costello, *Error Control Coding*, 2^a ed.

La Tabla 3 reproduce numéricamente la misma traza.

Tabla 3: Contenido del registro del codificador, ciclo a ciclo, para $\mathbf{u} = (1011)$ y $g(X) = 1 + X + X^3$.

Ciclo	Entrada	Bit del mensaje	Registro ($b_0 b_1 b_2$)
0	—	—	000
1	1	u_3	110
2	1	u_2	101
3	0	u_1	100
4	1	u_0	100

El resultado coincide con el cálculo directo: $X^3u(X) = X^3 + X^5 + X^6$ y, reduciendo módulo $g(X) = 1 + X + X^3$ mediante $X^3 \equiv 1 + X$, se obtiene $b(X) = 1$, es decir $\mathbf{b} = (100)$.

4. Cálculo del síndrome

Sea $r(X)$ el polinomio asociado a la palabra recibida. Dividiéndolo por el polinomio generador,

$$r(X) = a(X)g(X) + s(X), \quad \deg s(X) < n - k \quad (17)$$

El resto $s(X)$ se denomina *polinomio síndrome* (syndrome polynomial). Por el Teorema 2, $s(X) = 0$ si y sólo si $r(X)$ es una palabra de código. Escribiendo $r(X) = v(X) + e(X)$, donde $e(X)$ es el patrón de error, y usando que $g(X)$ divide a $v(X)$, se obtiene

$$s(X) = R_{g(X)}\{r(X)\} = R_{g(X)}\{e(X)\} \quad (18)$$

esto es, la propiedad fundamental ya conocida de los códigos lineales: **el síndrome depende sólo del patrón de error**. Lo nuevo es el modo de calcularlo. El síndrome de un código lineal genérico requiere el producto $\mathbf{r}H^T$; el de un código cíclico se obtiene con el mismo registro de $n - k$ etapas del codificador, alimentado ahora con la palabra recibida. La Figura 5 muestra el circuito.

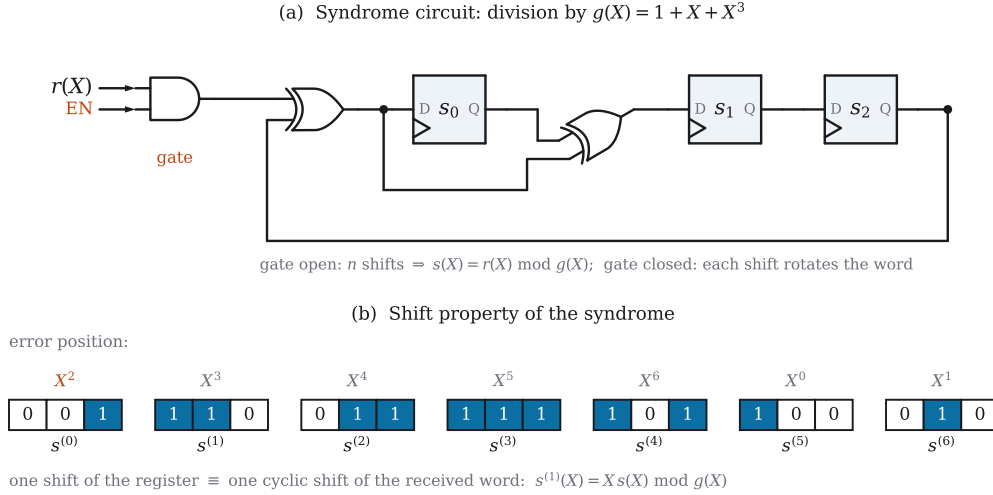


Figura 5: Arriba, el circuito divisor que calcula el síndrome: es el registro del codificador alimentado con $r(X)$; al cabo de n ciclos su contenido es $s(X)$. Abajo, la propiedad de desplazamiento: cada desplazamiento cíclico de la palabra recibida corresponde a un ciclo de reloj adicional del registro de síndrome, sin necesidad de recargarlo. El circuito del panel superior está adaptado de la Figura 5.5 de Lin & Costello, *Error Control Coding*, 2ª ed.

4.1. La propiedad de desplazamiento

La propiedad siguiente es la que hace posible la decodificación práctica de los códigos cíclicos, y merece un enunciado propio.

Teorema 5 (Teorema 5.8). *Sea $s(X)$ el síndrome de la palabra recibida $r(X)$ y sea $r^{(1)}(X)$ su desplazamiento cíclico. Entonces el síndrome de $r^{(1)}(X)$ es $s^{(1)}(X) = R_{g(X)}\{Xs(X)\}$, es decir, el resto de dividir $Xs(X)$ por $g(X)$.*

Demostración. Por (4), $r^{(1)}(X) = Xr(X) + r_{n-1}(X^n + 1)$. Escribiendo $X^n + 1 = h(X)g(X)$ y $r(X) = a(X)g(X) + s(X)$ se obtiene

$$r^{(1)}(X) = Xs(X) + [Xa(X) + r_{n-1}h(X)]g(X) \quad (19)$$

Como el segundo sumando es múltiplo de $g(X)$, el resto de $r^{(1)}(X)$ módulo $g(X)$ coincide con el resto de $Xs(X)$ módulo $g(X)$.

El significado operativo del Teorema 5 es el siguiente. El resto de $X s(X)$ módulo $g(X)$ es precisamente lo que produce el registro de síndrome cuando se le aplica un ciclo de reloj adicional con la entrada inhibida. Por lo tanto, **el decodificador puede recorrer los n desplazamientos cíclicos de la palabra recibida sin volver a dividir**: le basta con seguir haciendo funcionar el mismo registro. Iterando,

$$s^{(i)}(X) = R_{g(X)}\{X^i s(X)\} \quad (20)$$

La Tabla 4 lista los síndromes de los siete patrones de error simple del código $(7, 4)$ y permite verificar la propiedad: el síndrome de X^i es el desplazamiento de orden i del síndrome de X^0 .

Tabla 4: Síndromes de los errores simples del código $(7, 4)$ generado por $g(X) = 1 + X + X^3$. Obsérvese que los siete síndromes no nulos aparecen exactamente una vez, lo que confirma la capacidad de corrección de un error.

Error $e(X)$	Posición	Síndrome $s = (s_0 s_1 s_2)$
1	0	1 0 0
X	1	0 1 0
X^2	2	0 0 1
X^3	3	1 1 0
X^4	4	0 1 1
X^5	5	1 1 1
X^6	6	1 0 1

4.2. Detección de errores

Un código cíclico (n, k) detecta todo patrón de error que no sea una palabra de código, igual que cualquier código lineal. La estructura cíclica permite, sin embargo, caracterizar con precisión el comportamiento frente a un tipo de error de gran importancia práctica.

Definición. Un *error en ráfaga* (burst error) de longitud l es un patrón de error cuyas componentes no nulas están confinadas a l posiciones consecutivas, siendo la primera y la última de ellas no nulas. Polinómicamente, $e(X) = X^i B(X)$ con $\deg B(X) = l - 1$ y $B_0 = 1$.

Teorema 6 (Teorema 5.9). *Un código cíclico (n, k) detecta toda ráfaga de longitud $l \leq n - k$, incluidas las ráfagas que dan la vuelta por el extremo del bloque.*

Demostración. Si la ráfaga no fuera detectada, $e(X) = X^i B(X)$ sería una palabra de código y, por el Teorema 2, $g(X)$ dividiría a $X^i B(X)$. Como $g_0 = 1$, los polinomios $g(X)$ y X^i son coprimos, de modo que $g(X)$ debería dividir a $B(X)$. Pero $\deg B(X) = l - 1 < n - k = \deg g(X)$ y $B(X) \neq 0$, lo que es imposible.

La demostración no usa en ningún momento que las l posiciones consecutivas estén

contenidas dentro del bloque: la factorización $e(X) = X^i B(X)$ vale igualmente si la ráfaga se interrumpe en el extremo de mayor orden y continúa en el de menor orden. Este segundo tipo de patrón recibe el nombre de *ráfaga cíclica* (end-around burst) y queda cubierto por el mismo enunciado. Es una consecuencia directa de la estructura de anillo introducida en la Sección 1: sobre el anillo de n posiciones no hay extremos, y una ráfaga cíclica es una ráfaga ordinaria vista desde otro origen. Así, el patrón

$$\mathbf{e} = (1\ 0\ 1\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 1\ 1\ 0\ 1) \quad (21)$$

es una ráfaga cíclica de longitud 7, aunque leído de izquierda a derecha aparente ser un patrón disperso de peso 5.

Para ráfagas más largas la detección deja de ser segura, pero falla sólo en una fracción muy pequeña de los casos. Un recuento de los patrones de ráfaga que son palabras de código conduce a:

- la fracción de ráfagas de longitud $l = n - k + 1$ no detectadas es $2^{-(n-k-1)}$;
- la fracción de ráfagas de longitud $l > n - k + 1$ no detectadas es $2^{-(n-k)}$.

Estas cifras justifican por sí solas el uso masivo de códigos cíclicos como códigos detectores. Un código con $n - k = 32$ bits de redundancia deja pasar, en el peor caso, una ráfaga de cada 4 294 967 296, con independencia de la longitud del bloque. La Figura 10, en la sección dedicada a los códigos acortados, presenta gráficamente este comportamiento.

5. Decodificación

5.1. El decodificador de Meggitt

La decodificación de un código lineal genérico exige almacenar la correspondencia entre los 2^{n-k} síndromes y sus líderes de coset. La propiedad de desplazamiento permite reemplazar esa tabla por un circuito combinacional mucho más pequeño, mediante el siguiente razonamiento.

Supóngase que se dispone de un circuito capaz de responder a una única pregunta: *dado el síndrome corriente, ¿el patrón de error correspondiente tiene un uno en la posición de mayor orden X^{n-1} ?* Con ese circuito alcanza para corregir todas las posiciones. En efecto, si se desplaza cíclicamente la palabra recibida, la posición $n - 1 - i$ pasa a ocupar el lugar $n - 1$; y por el Teorema 5 el síndrome del desplazamiento se obtiene con un solo ciclo de reloj. Recorriendo los n desplazamientos, cada posición pasa por el lugar $n - 1$ exactamente una vez y es examinada por el mismo circuito.

Formalmente, sea S_{n-1} el conjunto de los síndromes de todos los patrones de error corregibles $e(X)$ con $e_{n-1} = 1$. El *decodificador de Meggitt* opera así:

1. Se calcula el síndrome de $r(X)$ y se lo carga en el registro de síndrome; en paralelo, $r(X)$ se carga en un buffer de n etapas.
2. En el paso i ($i = 0, 1, \dots, n - 1$) la lógica combinacional evalúa si el contenido del registro pertenece a S_{n-1} . Si pertenece, la posición $n - 1 - i$ está errada: el bit correspondiente se corrige a la salida del buffer y el efecto del error se elimina del registro de síndrome.

3. Se aplica un ciclo de reloj al registro de síndrome y se repite.

Al cabo de n ciclos el buffer ha entregado la palabra corregida. Si el registro de síndrome queda en cero, la decodificación fue exitosa; si no, se detectó un patrón de error no corregible. La Figura 6 presenta el diagrama en bloques.

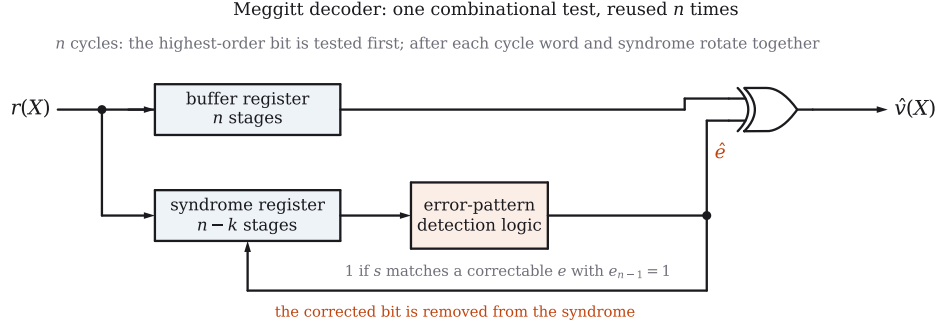


Figura 6: Decodificador de Meggitt. El registro de síndrome, de $n - k$ etapas, y el buffer, de n etapas, avanzan al mismo ritmo; la lógica combinacional decide, en cada ciclo, si la posición que está saliendo del buffer se encuentra errada. Abajo, el conjunto de detección del código $(7, 4)$ para $t = 1$: un único síndrome, de modo que toda la lógica se reduce a una compuerta AND de tres entradas. Adaptada de la Figura 5.8 de Lin & Costello, *Error Control Coding*, 2ª ed.; el conjunto de detección corresponde al Ejemplo 5.9 y a la Tabla 5.4 de esa obra

La ganancia es considerable. La lógica combinacional debe reconocer únicamente los síndromes de los patrones de error corregibles **con un uno en la posición de mayor orden**, que son

$$|S_{n-1}| = \sum_{i=0}^{t-1} \binom{n-1}{i} \quad (22)$$

en lugar de los $\sum_{i=0}^t \binom{n}{i}$ líderes de coset del standard array. Para el código $(7, 4)$ con $t = 1$ el conjunto tiene un solo elemento –el síndrome (101) del error X^6 – y el decodificador completo se reduce a un registro de tres etapas, un buffer de siete y una compuerta AND. El contraste con la tabla de ocho entradas del standard array es modesto en este ejemplo por lo pequeño del código, pero crece exponencialmente con $n - k$.

La Figura 7 sigue el proceso completo sobre un caso concreto y permite ver el mecanismo en funcionamiento.

	syndrome register	buffer register	correction
initial	0 0 1	1 0 1 1 0 1 1	0
shift 1	1 1 0	1 1 0 1 1 0 1	0
shift 2	0 1 1	1 1 1 0 1 1 0	0
shift 3	1 1 1	0 1 1 1 0 1 1	0
shift 4	1 0 1	1 0 1 1 1 0 1	1 error corrected
shift 5	0 0 0	0 1 0 1 1 1 0	0
shift 6	0 0 0	0 0 1 0 1 1 1	0
shift 7	0 0 0	1 0 0 1 0 1 1	

$\mathbf{v} = (1001011)$ transmitted, $\mathbf{r} = (1011011)$ received (single error at X^2)
detection pattern (1 0 1) reached after 4 shifts; the syndrome then returns to zero

Figura 7: Proceso de corrección del decodificador de Meggitt para el código (7,4). Se transmite $\mathbf{v} = (1001011)$ y se recibe $\mathbf{r} = (1011011)$, con un error en la posición X^2 . El registro de síndrome parte de (001) y alcanza el patrón de detección (101) tras cuatro desplazamientos, justo cuando el dígito errado ocupa la etapa de mayor orden del buffer; corregido el error, el síndrome vuelve a cero y los desplazamientos restantes no producen ninguna otra corrección. Reproduce el Ejemplo 5.9 y la Figura 5.10 de Lin & Costello, *Error Control Coding*, 2ª ed.

Hay un costo que la comparación con el standard array no muestra. El decodificador de Meggitt trabaja **en serie**: examina un dígito por ciclo de reloj y necesita n ciclos para entregar la palabra completa, mientras que un decodificador combinacional construido sobre la tabla de síndromes produce la corrección en un solo paso. La estructura cíclica no ofrece, entonces, una mejora en todos los frentes a la vez: cambia área por tiempo. En el caso del código (7, 4) el circuito cíclico es apreciablemente más simple que el decodificador combinacional de la Unidad 4, pero tarda siete veces más. Se trata de un compromiso entre velocidad y simplicidad que reaparece en toda la ingeniería de decodificadores.

El decodificador de Meggitt sigue siendo, de todos modos, un procedimiento cuya complejidad crece con t : para t grande el número de síndromes a reconocer se vuelve inmanejable. Es un decodificador excelente para códigos con capacidad correctora pequeña, y la base conceptual de los decodificadores algebraicos más elaborados.

5.2. Decodificación por captura de errores

Existe un caso particular en el que el reconocimiento de síndromes se vuelve trivial. Supóngase que todos los errores están confinados a las $n - k$ posiciones de paridad, es decir, que $\deg e(X) < n - k$. Entonces la división de $e(X)$ por $g(X)$ tiene cociente nulo y

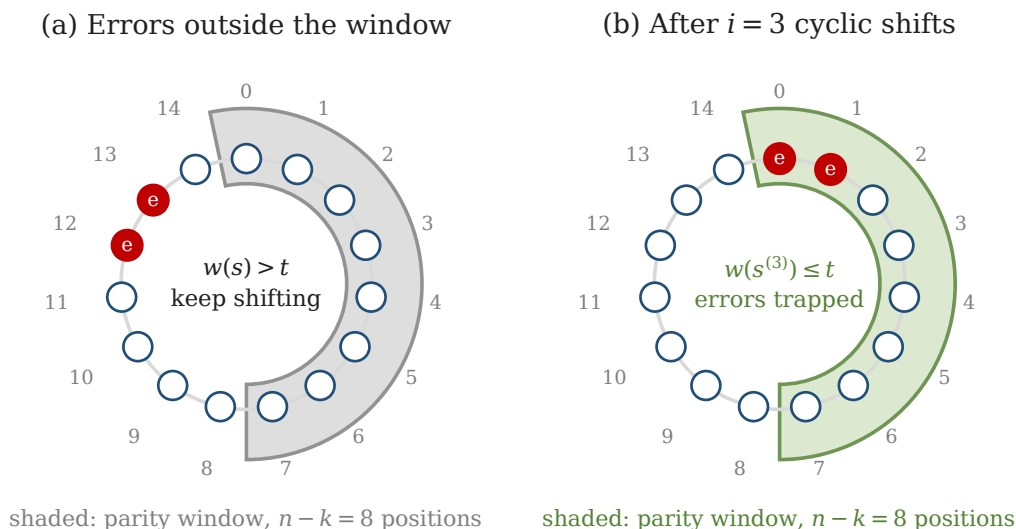
$$s(X) = R_{g(X)}\{e(X)\} = e(X) \quad (23)$$

el síndrome es el patrón de error. No hace falta ninguna tabla ni ninguna lógica: basta con sumar el contenido del registro de síndrome a las posiciones de paridad de la palabra recibida.

La *decodificación por captura de errores* (error-trapping decoding) generaliza esta observación mediante la propiedad de desplazamiento. Aunque los errores no estén inicialmente en las posiciones de paridad, si están agrupados es posible que algún desplazamiento cíclico los lleve a esa ventana. El procedimiento es:

1. Se calcula $s(X)$ y se observa su peso de Hamming $w[s]$.
2. Si $w[s] \leq t$, se declara que los errores están capturados: el contenido del registro es el patrón de error, y se corrige.
3. Si no, se aplica un ciclo de reloj al registro –lo que equivale a desplazar cíclicamente la palabra recibida– y se repite, hasta un máximo de n veces.

La Figura 8 ilustra el mecanismo sobre el anillo de posiciones.



trapped = the syndrome register content $s^{(i)}$ is the error pattern itself

Figura 8: Captura de errores. Las $n - k$ posiciones de paridad forman una ventana sobre el anillo. Mientras los errores caigan fuera de la ventana, el peso del síndrome supera a t y el decodificador sigue desplazando; cuando los errores entran en la ventana, el síndrome coincide con el patrón de error. La disposición de las posiciones sobre un anillo sigue a la Figura 5.16 de Lin & Costello, *Error Control Coding*, 2ª ed.

El criterio de parada merece una justificación, porque no es evidente que un síndrome liviano garantice que los errores estén efectivamente capturados. Sea $e(X) = X^j B(X)$ un patrón con t errores o menos confinado a $n - k$ posiciones consecutivas, de modo que $\deg B(X) \leq n - k - 1$, y sea $s(X)$ el síndrome corriente. La suma $s(X) + X^j B(X)$ es múltiplo de $g(X)$ y, por lo tanto, una palabra de código. Si se diera $w[s(X)] \leq t$ con $s(X) \neq X^j B(X)$, esa palabra sería no nula y de peso menor que $2t + 1$, lo que contradice que el código corrija t errores y tenga por ello distancia

mínima al menos $2t + 1$. En consecuencia, el peso del síndrome cae a t o menos **si y sólo si** los errores están capturados, y el test de peso es una condición necesaria y suficiente.

La limitación del método es igualmente clara: sólo funciona si existe una ventana de $n - k$ posiciones consecutivas –en sentido cíclico– que contenga a todos los errores. Para errores aleatorios dispersos la condición falla, y el decodificador declara fallo de decodificación. Para errores en ráfaga, en cambio, la condición se cumple casi siempre, y por eso la captura de errores es el método de elección en los códigos correctores de ráfagas. Existen variantes que amplían el conjunto de patrones capturables a costa de lógica adicional. La más conocida es la de Kasami, que admite patrones en los que la mayoría de los errores cae dentro de la ventana de $n - k$ posiciones y unos pocos quedan fuera: se elige un conjunto de *polinomios de cobertura* $\{\phi_j(X)\}$ y se testea el peso de $s^{(i)}(X) + \rho_j(X)$ contra el umbral $t - w[\phi_j(X)]$, lo que exige tantas compuertas de umbral como polinomios de cobertura. Para el Golay (23, 12) bastan tres, con $\phi_1(X) = 0$, $\phi_2(X) = X^5$ y $\phi_3(X) = X^6$. El método sigue siendo practicable sólo en códigos relativamente cortos y de tasa baja: cuando n y t crecen, la cantidad de compuertas de umbral se vuelve prohibitiva.

Conviene cuantificar la limitación sobre casos concretos, porque el resultado depende críticamente de la relación entre n y $n - k$. Considérese primero el código cíclico (15, 7) generado por $g(X) = 1 + X^4 + X^6 + X^7 + X^8$, de distancia mínima 5 y capacidad correctora $t = 2$. Sobre el anillo de 15 posiciones, dos posiciones cualesquiera distan a lo sumo siete lugares por el arco más corto, de modo que siempre quedan contenidas en una ventana de $n - k = 8$ posiciones consecutivas. Los 105 patrones de error doble son entonces **capturados sin excepción**: para este código la captura de errores no es una heurística sino un decodificador completo.

El código de Golay (23, 12), con $t = 3$ y $n - k = 11$, muestra el caso opuesto. El patrón de peso dos

$$e(X) = X^{11} + X^{22} \quad (24)$$

tiene sus dos errores separados por once y doce posiciones sobre el anillo de 23, según el arco que se recorra. Una ventana de once posiciones consecutivas abarca diferencias de a lo sumo diez, de modo que ningún desplazamiento cíclico logra confinar ambos errores: el patrón **nunca puede ser capturado**, por más que se haga girar el registro. Otro tanto ocurre con patrones triples como $e(X) = X^5 + X^{11} + X^{22}$. La captura simple aplicada al Golay pierde así una parte de la capacidad correctora del código, y por eso su decodificación práctica exige los refinamientos descritos más arriba.

La comparación entre ambos ejemplos deja la regla de diseño: la captura de errores es efectiva cuando la ventana $n - k$ es grande respecto de la longitud n , es decir, en códigos de tasa baja o moderada, y se vuelve ineficaz en los códigos largos y de alta tasa, donde la ventana cubre una fracción pequeña del anillo. También lo es –y es su aplicación principal– en los códigos correctores de ráfagas, donde los errores vienen agrupados por construcción.

6. Familias de códigos cíclicos

6.1. Códigos de Hamming cíclicos

Los códigos de Hamming de la Unidad 4 admiten una construcción cíclica de gran elegancia.

Proposición. *Sea $p(X)$ un polinomio primitivo de grado m sobre $GF(2)$. Entonces $p(X)$ genera un código cíclico de longitud $n = 2^m - 1$ y dimensión $k = n - m$, cuya distancia mínima es 3: un código de Hamming.*

El argumento se apoya en la Unidad 2. Si α es una raíz de $p(X)$, la primitividad de $p(X)$ significa que α tiene orden $2^m - 1$, de modo que α es raíz de $X^n + 1$ y, por lo tanto, $p(X)$ divide a $X^n + 1$: por el Teorema 4 el código es cíclico. Además, $v(X)$ es palabra de código si y sólo si $v(\alpha) = 0$, lo que equivale a la condición matricial $\mathbf{v}H^T = \mathbf{0}$ con

$$H = \begin{pmatrix} 1 & \alpha & \alpha^2 & \cdots & \alpha^{n-1} \end{pmatrix} \quad (25)$$

Las columnas de H son las $2^m - 1$ potencias distintas de α , es decir, todos los elementos no nulos de $GF(2^m)$: exactamente la matriz de paridad de un código de Hamming. Como las columnas son no nulas y distintas de α , no hay dos ni una sola que sumen cero, y sí hay tres que lo hacen; luego $d_{\min} = 3$.

La construcción es notable porque reduce el diseño de un código de Hamming de longitud arbitraria a la elección de un polinomio primitivo, y su implementación a un registro de m etapas. Para $m = 3$, con $p(X) = 1 + X + X^3$, se obtiene el código (7, 4) usado como ejemplo en esta unidad; para $m = 4$, con $p(X) = 1 + X + X^4$, el código (15, 11) de la Tabla 1.

Multiplicando por $(1 + X)$ el polinomio generador se obtiene un código cíclico de longitud $2^m - 1$ y $m + 1$ bits de paridad, cuya distancia mínima es 4: corrige un error y detecta dos simultáneamente, es decir, la familia SEC-DED de la Unidad 4 en versión cíclica. La razón es transparente en términos polinómicos: las palabras del código generado por $(X + 1)p(X)$ son exactamente las palabras de peso par del código de Hamming, porque un polinomio de peso impar no admite $X + 1$ como factor. Como el código original tiene distancia 3, el nuevo tiene distancia al menos 4, y puede probarse que la alcanza.

Estos códigos se emplean con frecuencia como códigos detectores. Para el de distancia 4, la probabilidad de error no detectado sobre el canal binario simétrico satisface la cota

$$P_u(E) \leq 2^{-(n-k)} = 2^{-(m+1)} \quad (26)$$

que es el comportamiento deseable de un buen código detector: la probabilidad de dejar pasar un error queda acotada por la redundancia empleada, con independencia de la calidad del canal.

6.2. El código de Golay

El código de Golay (23, 12), presentado en la Unidad 4 por su distribución de pesos y su carácter de código perfecto, es también un código cíclico. La factorización

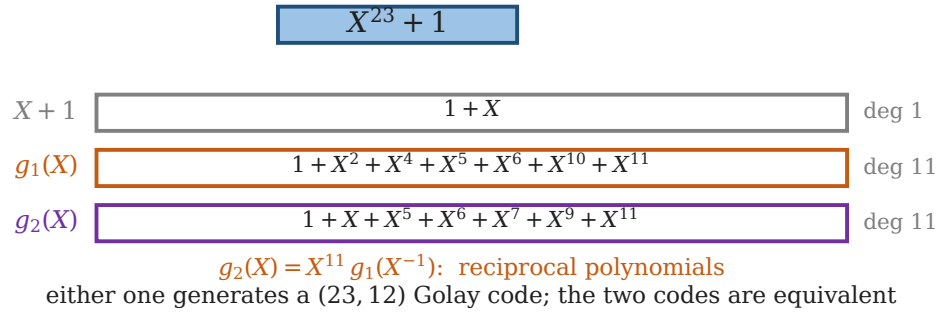
$$X^{23} + 1 = (X + 1) g_1(X) g_2(X) \quad (27)$$

con

$$\begin{aligned} g_1(X) &= 1 + X^2 + X^4 + X^5 + X^6 + X^{10} + X^{11} \\ g_2(X) &= 1 + X + X^5 + X^6 + X^7 + X^9 + X^{11} \end{aligned} \quad (28)$$

exhibe dos polinomios irreducibles de grado 11, recíprocos entre sí, cada uno de los cuales genera un código (23, 12). Los dos códigos son equivalentes; ambos tienen distancia mínima 7 y corrigen hasta tres errores.

(a) $X^{23} + 1$ and the two Golay generators



(b) Perfect packing of the (23, 12) code

2^{12} spheres $\times$ 2^{11} words each = 2^{23} : the spheres tile the whole space

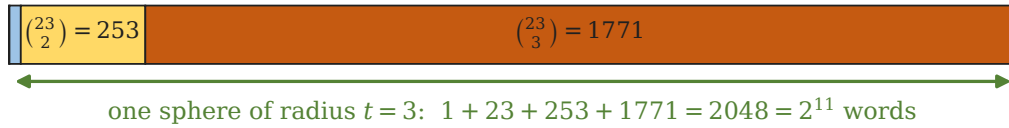


Figura 9: Arriba, la factorización de $X^{23} + 1$ y los dos polinomios generadores del código de Golay. Abajo, la verificación del carácter perfecto: las 2^{12} esferas de radio 3 contienen exactamente 2^{23} palabras, es decir, la totalidad del espacio.

La condición de código perfecto se verifica por conteo directo:

$$2^{12} \sum_{i=0}^3 \binom{23}{i} = 2^{12} (1 + 23 + 253 + 1771) = 2^{12} \cdot 2^{11} = 2^{23} \quad (29)$$

Las esferas de radio $t = 3$ centradas en las palabras de código recubren el espacio V_{23} sin solapamiento y sin dejar huecos. El código de Golay (23, 12) y los códigos de Hamming son, junto con los códigos de repetición de longitud impar, los únicos códigos binarios perfectos no triviales que existen.

Su enumerador de pesos, ya obtenido en la Unidad 4, es

$$A(z) = 1 + 253z^7 + 506z^8 + 1288z^{11} + 1288z^{12} + 506z^{15} + 253z^{16} + z^{23} \quad (30)$$

y de él se sigue que, usado como código detector sobre el canal binario simétrico, su probabilidad de error no detectado satisface $P_u(E) \leq 2^{-11}$.

Su interés en esta unidad es de otro orden. Al ser cíclico, el Golay admite un codificador de once etapas y un cálculo de síndrome por división, lo que resuelve la implementación del codificador; la dificultad está del lado del decodificador. Como se vio, la captura simple no alcanza –hay patrones de peso dos que nunca se capturan–, de modo que su decodificación completa hasta $t = 3$ requiere un refinamiento. Los dos procedimientos clásicos son el decodificador de Kasami, con sus tres compuertas de umbral, y el llamado decodificador de búsqueda sistemática, que invierte uno a uno los doce dígitos de información y aplica captura ordinaria tras cada inversión: en todo patrón de tres errores o menos hay siempre al menos un error que, corregido, deja a los restantes dentro de once posiciones consecutivas. El primero es más rápido; el segundo usa una sola compuerta de umbral a costa de una lógica de temporización más compleja.

6.3. Códigos cíclicos acortados

Las longitudes de los códigos cíclicos no son arbitrarias: quedan determinadas por las factorizaciones de $X^n + 1$, y en el caso de los códigos primitivos valen $2^m - 1$. Las aplicaciones, en cambio, imponen longitudes fijadas por el formato de las tramas, que rara vez coinciden con esos valores. El *acortamiento* (shortening) resuelve el desajuste.

Dado un código cíclico (n, k) , se consideran únicamente las palabras cuyas l posiciones de información de mayor orden son nulas. Como esas posiciones son siempre cero, no hace falta transmitir las, y se obtiene un código $(n - l, k - l)$. La construcción tiene tres consecuencias:

- El conjunto de palabras del código acortado es un **subconjunto** del original, de modo que su distancia mínima es mayor o igual que la de aquel: el acortamiento nunca empeora la capacidad correctora.
- El código acortado, en general, **ya no es cíclico**. Un desplazamiento cíclico puede llevar un uno a una de las posiciones suprimidas.
- Los circuitos de codificación y de cálculo de síndrome del código original **siguen sirviendo sin modificación**, ya que las posiciones anuladas no alteran las divisiones. Ésta es la razón práctica del método.

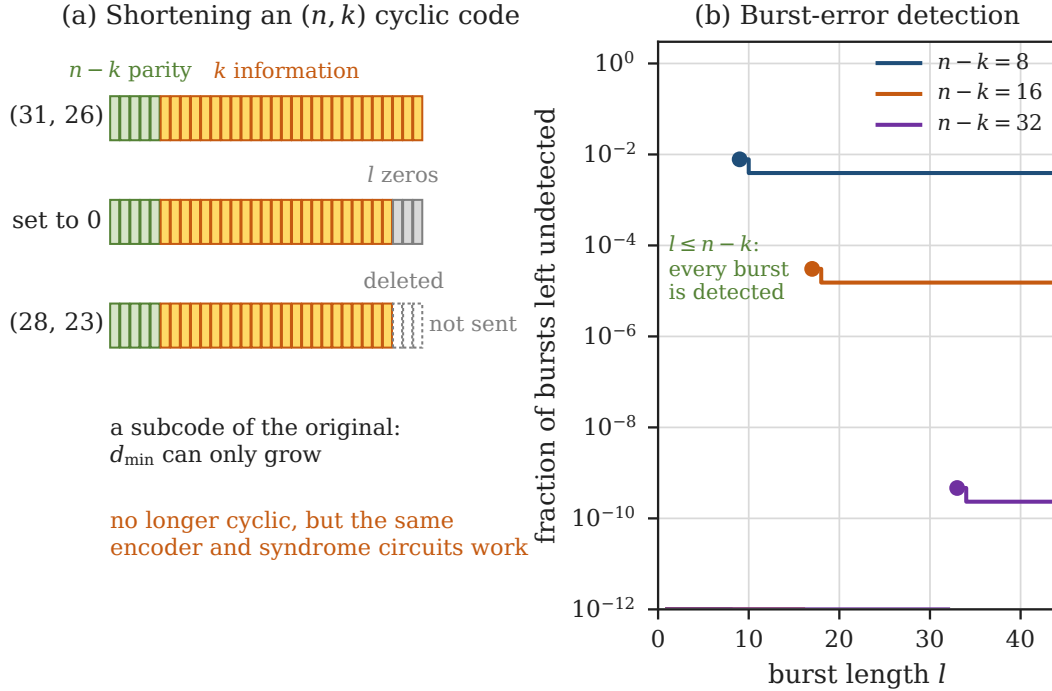


Figura 10: A la izquierda, el acortamiento de un código cíclico: se anulan l posiciones de información y se las suprime de la transmisión. A la derecha, la fracción de ráfagas no detectadas en función de su longitud, para tres valores de redundancia; hasta $l = n - k$ la detección es segura. El acortamiento ilustrado reproduce el Ejemplo 5.12 de Lin & Costello, *Error Control Coding*, 2ª ed., que obtiene el código $(28, 23)$ suprimiendo tres posiciones del Hamming cíclico $(31, 26)$ generado por $g(X) = 1 + X^2 + X^5$.

La tercera afirmación merece una precisión, porque el reaprovechamiento de los circuitos no es del todo gratuito del lado del decodificador. Si la palabra recibida se ingresa al registro de síndrome por el extremo derecho, lo que equivale a premultiplicarla por X^{n-k} , el contenido del registro al cabo de $n - l$ ciclos no es todavía el síndrome que corresponde al primer dígito a decodificar: hacen falta l desplazamientos adicionales. Para l grande esa espera es inaceptable, y se la elimina de dos maneras equivalentes. La primera consiste en multiplicar la palabra recibida por el polinomio

$$\rho(X) = R_{g(X)}\{X^{n-k+l}\} \quad (31)$$

mientras se la divide por $g(X)$, lo que se realiza agregando al registro un segundo juego de conexiones de entrada dadas por los coeficientes de $\rho(X)$. Para el código $(28, 23)$ del ejemplo anterior resulta $\rho(X) = R_{g(X)}\{X^8\} = 1 + X^2 + X^3$. La segunda consiste en dejar el registro como está y rediseñar el circuito de detección para que reconozca el síndrome desplazado en lugar del original. En ambos casos el costo es una modificación menor del cableado, no un circuito nuevo.

La aplicación más difundida de los códigos cíclicos acortados es la detección de errores en tramas de datos. Los códigos así empleados reciben el nombre de *cyclic redundancy check* (CRC) y sus polinomios generadores están normalizados. La Tabla 5 recoge los de uso más frecuente.

Tabla 5: Polinomios generadores de los CRC normalizados que recoge Lin & Costello, *Error Control Coding*, 2ª ed. Todos ellos se construyen como $(X+1)p(X)$, con $p(X)$ primitivo o producto de primitivos.

Norma	Polinomio generador
CCITT X-25	$g(X) = (X+1)(X^{15} + X^{14} + X^{13} + X^{12} + X^4 + X^3 + X^2 + X + 1)$ $= X^{16} + X^{12} + X^5 + 1$
ANSI	$g(X) = (X+1)(X^{15} + X + 1) = X^{16} + X^{15} + X^2 + 1$
IBM-SDLC	$g(X) = (X+1)^2(X^{14} + X^{13} + X^{12} + X^{10} + X^8 + X^6 + X^5 + X^4 + X^3 + X + 1)$ $= X^{16} + X^{15} + X^{13} + X^7 + X^4 + X^2 + X + 1$
IEC TC57	$g(X) = (X+1)^2(X^{14} + X^{10} + X^9 + X^8 + X^5 + X^3 + X^2 + X + 1)$ $= X^{16} + X^{14} + X^{11} + X^8 + X^6 + X^5 + X^4 + 1$
IEEE 802.3	$g(X) = X^{32} + X^{26} + X^{23} + X^{22} + X^{16} + X^{12} + X^{11}$ $+ X^{10} + X^8 + X^7 + X^5 + X^4 + X^2 + X + 1$

El rasgo común de todos ellos es el factor $(X+1)$, que garantiza que toda palabra de código tenga peso par y, en consecuencia, que se detecte cualquier patrón de error de peso impar. El otro factor es primitivo, lo que asegura que el código sin acortar sea un Hamming de distancia 4 y que, por lo tanto, se detecte además todo patrón de uno, dos o tres errores. A esto se suma la detección de ráfagas del Teorema 6: un CRC de 32 bits detecta toda ráfaga de longitud hasta 32 y deja pasar, en el peor caso, una de cada 2^{32} , con independencia de la longitud de la trama.

6.4. Códigos producto cíclicos

La Unidad 4 introdujo el código producto de dos códigos C_1 y C_2 , obtenido disponiendo la información en un arreglo rectangular y codificando filas y columnas. El resultado es un código (n_1n_2, k_1k_2) de distancia mínima d_1d_2 . Cabe preguntarse si esa construcción preserva la ciclicidad, y la respuesta es afirmativa bajo una condición aritmética simple.

Proposición. *Si C_1 y C_2 son cíclicos de longitudes n_1 y n_2 con $\text{mcd}(n_1, n_2) = 1$, entonces su código producto es cíclico de longitud n_1n_2 , siempre que las posiciones del arreglo se recorran en el orden dictado por el teorema chino del resto.*

El orden de lectura consiste en asignar a la posición m del vector de longitud n_1n_2 la celda $(m \bmod n_1, m \bmod n_2)$ del arreglo. La coprimalidad garantiza, por el teorema chino del resto, que la correspondencia sea biunívoca, y hace que un desplazamiento del vector corresponda simultáneamente a un desplazamiento de filas y a uno de columnas, ambos admisibles por hipótesis. Leído sobre el arreglo, ese orden recorre diagonales a 45 grados: se parte de una esquina, se avanza en diagonal hasta salir del arreglo y se reingresa por el borde opuesto, hasta cubrir las n_1n_2 celdas.

El polinomio generador del código producto puede escribirse en forma cerrada. Como n_1 y n_2 son coprimos, existen enteros a y b tales que $an_1 + bn_2 = 1$, y entonces

$$g(X) = \text{mcd}[X^{n_1 n_2} + 1, g_1(X^{b_{n_2}}) g_2(X^{a_{n_1}})] \quad (32)$$

donde $g_1(X)$ y $g_2(X)$ son los generadores de C_1 y C_2 . La expresión análoga con los polinomios de paridad da el $h(X)$ del código producto. El resultado es notable: un código cuya construcción es bidimensional queda descrito, al final, por un único polinomio generador, y hereda toda la maquinaria de esta unidad.

Del lado del decodificador la ventaja es igualmente concreta. La palabra recibida puede reordenarse como arreglo y decodificarse por filas y por columnas con los decodificadores de C_1 y C_2 por separado, de modo que el equipamiento total es aproximadamente el de los dos códigos componentes, y no el que exigiría un código de longitud $n_1 n_2$ tratado en bloque.

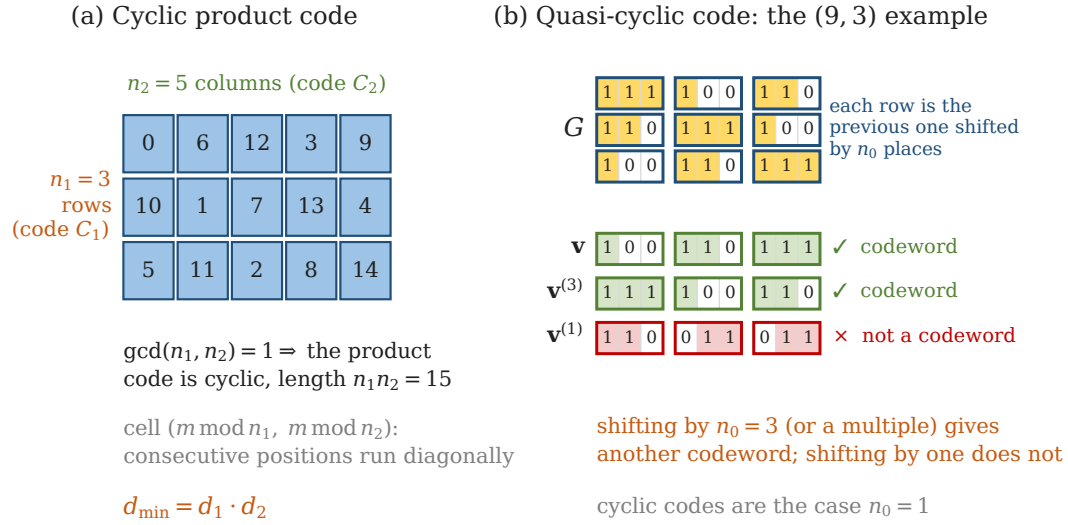


Figura 11: A la izquierda, el arreglo de un código producto de longitudes coprimas $n_1 = 3$ y $n_2 = 5$, con las posiciones numeradas según el orden del teorema chino del resto. A la derecha, la matriz generadora de un código quasi-cíclico, formada por bloques circulantes. El orden de transmisión en diagonal corresponde a la Figura 5.23 de Lin & Costello, *Error Control Coding*, 2ª ed.

El interés de esta construcción es doble: por un lado permite obtener códigos largos y de gran distancia a partir de códigos cortos y fáciles de decodificar; por otro, dota al código producto de la estructura cíclica, con las ventajas de implementación ya discutidas.

6.5. Códigos quasi-cíclicos

Una última generalización relaja la condición de ciclicidad.

Definición. Un código de bloque lineal se dice *quasi-cíclico* (quasi-cyclic) si existe un entero $n_0 \neq 1$ tal que el desplazamiento cíclico de toda palabra de código en n_0 posiciones –o en un múltiplo de n_0 – es también una palabra de código. Al entero n_0 se lo denomina *restricción de desplazamiento* (shifting constraint).

Los códigos cíclicos son el caso particular $n_0 = 1$: poseen simetría cíclica plena. Los quasi-cíclicos conservan sólo una simetría parcial, y esa relajación es precisamente lo que amplía el catálogo de longitudes y tasas disponibles. Puede probarse, además, que el dual de un código quasi-cíclico vuelve a ser quasi-cíclico.

El ejemplo mínimo es el código $(9, 3)$ generado por

$$G = \begin{pmatrix} 111 & 100 & 110 \\ 110 & 111 & 100 \\ 100 & 110 & 111 \end{pmatrix} \quad (33)$$

con restricción de desplazamiento $n_0 = 3$. Desplazando cualquiera de sus ocho palabras tres posiciones se obtiene otra palabra del código; desplazándola una o dos posiciones, en general no. La matriz generadora de un código quasi-cíclico $(m n_0, m k_0)$ puede llevarse siempre a una forma sistemática por bloques, en la que cada fila de bloques es el desplazamiento cíclico de la anterior, según se muestra en el panel derecho de la Figura 11. Cada bloque circulante queda especificado por su primera fila, de modo que la matriz completa se describe con muy pocos parámetros. Una palabra consta de m bloques de n_0 símbolos, cada uno con k_0 símbolos de información inalterados y $n_0 - k_0$ de paridad.

La estructura en bloques circulantes se traduce en implementaciones paralelas: los b bits de un bloque pueden procesarse simultáneamente, lo que resulta decisivo cuando se busca alto régimen binario. Esta familia reaparecerá en el curso al tratar los códigos LDPC, cuya variante quasi-cíclica es la que se emplea en los estándares modernos.

7. Síntesis

La unidad puede resumirse en cuatro afirmaciones.

Primera: un polinomio describe un código. Un código cíclico (n, k) queda completamente determinado por su polinomio generador $g(X)$, un divisor de $X^n + 1$ de grado $n - k$. Los $n - k + 1$ coeficientes de $g(X)$ reemplazan a los $k(n - k)$ bits de la submatriz de paridad.

Segunda: codificar es dividir. La palabra sistemática se obtiene como $v(X) = b(X) + X^{n-k}u(X)$ con $b(X) = R_{g(X)}\{X^{n-k}u(X)\}$, y la división se realiza con un registro de desplazamiento de $n - k$ etapas. La complejidad del codificador crece linealmente con la redundancia.

Tercera: el síndrome se desliza con la palabra. El Teorema 5 establece que el síndrome del desplazamiento cíclico de la palabra recibida se obtiene aplicando un ciclo de reloj al mismo registro de síndrome. Sobre esta propiedad se apoyan tanto el decodificador de Meggitt, que reduce el reconocimiento de 2^{n-k} síndromes al de un subconjunto mucho menor, como la decodificación por captura de errores, que en el caso favorable elimina por completo la necesidad de reconocer síndromes.

Cuarta: la construcción es sistemática, pero la elección importa. Factorizar $X^n + 1$ produce el catálogo completo de los códigos cíclicos de longitud n , y en ese catálogo conviven, para una misma redundancia, códigos de distancias mínimas muy diferentes. La teoría desarrollada en esta unidad no proporciona un criterio para elegir el mejor polinomio generador; sólo garantiza que todo divisor de $X^n + 1$ genera un código cíclico. El problema de construir polinomios generadores con distancia

mínima garantizada por diseño, y no verificada por enumeración, es el objeto de las unidades siguientes.

Referencias

- [1] S. Lin y D. J. Costello, *Error Control Coding*, 2^a ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2004.
- [2] E. Prange, “Cyclic error-correcting codes in two symbols”, Air Force Cambridge Research Center, Cambridge, MA, Informe técnico AFCRC-TN-57-103, sept. 1957.
- [3] W. W. Peterson y D. T. Brown, “Cyclic codes for error detection”, *Proceedings of the IRE*, vol. 49, n.º 1, pp. 228–235, ene. 1961.
- [4] J. E. Meggitt, “Error correcting codes and their implementation for data transmission systems”, *IRE Transactions on Information Theory*, vol. IT-7, n.º 4, pp. 234–244, oct. 1961.
- [5] T. Kasami, “A decoding procedure for multiple-error-correcting cyclic codes”, *IEEE Transactions on Information Theory*, vol. IT-10, n.º 2, pp. 134–139, abr. 1964.
- [6] F. J. MacWilliams, “Permutation decoding of systematic codes”, *Bell System Technical Journal*, vol. 43, n.º 1, pp. 485–505, ene. 1964.
- [7] M. J. E. Golay, “Notes on digital coding”, *Proceedings of the IRE*, vol. 37, p. 657, jun. 1949.
- [8] P. Fire, “A class of multiple-error-correcting binary codes for non-independent errors”, Sylvania Electronic Defense Laboratory, Mountain View, CA, Informe RSL-E-2, 1959.
- [9] H. O. Burton y E. J. Weldon, “Cyclic product codes”, *IEEE Transactions on Information Theory*, vol. IT-11, n.º 3, pp. 433–439, jul. 1965.
- [10] R. L. Townsend y E. J. Weldon, “Self-orthogonal quasi-cyclic codes”, *IEEE Transactions on Information Theory*, vol. IT-13, n.º 2, pp. 183–195, abr. 1967.
- [11] C. L. Chen, W. W. Peterson y E. J. Weldon, “Some results on quasi-cyclic codes”, *Information and Control*, vol. 15, n.º 5, pp. 407–423, 1969.
- [12] W. W. Peterson y E. J. Weldon, *Error-Correcting Codes*, 2^a ed. Cambridge, MA: MIT Press, 1972.
- [13] F. J. MacWilliams y N. J. A. Sloane, *The Theory of Error-Correcting Codes*. Amsterdam: North-Holland, 1977.
- [14] R. E. Blahut, *Algebraic Codes for Data Transmission*. Cambridge: Cambridge University Press, 2003.
- [15] S. B. Wicker, *Error Control Systems for Digital Communication and Storage*. Englewood Cliffs, NJ: Prentice Hall, 1995.
- [16] E. Sanvicente, *Understanding Error Control Coding*. Cham: Springer, 2019.
- [17] G. Castagnoli, S. Bräuer y M. Herrmann, “Optimization of cyclic redundancy-check codes with 24 and 32 parity bits”, *IEEE Transactions on Communications*, vol. 41, n.º 6, pp. 883–892, jun. 1993.
- [18] Optical Internetworking Forum, *Implementation Agreement 400ZR*, OIF-400ZR-02.0, 2022.